Portable Scientific Computing Across ML Accelerators with Execution-Grounded Agents

Abstract—ML accelerators offer substantial performance and efficiency gains for linear-algebra-heavy workloads; however, their ML-framework-centric programming model constrains their use to mostly AI workloads, putting them out of reach of domain scientists.

We present a multi-agent pipeline that aims to democratize access to ML accelerators for scientific computing by automatically porting linear-algebra-heavy kernels from C to PyTorch, then profiling the resulting models across heterogeneous platforms, including Nvidia GPUs, Groq LPU, GraphCore IPU, and Cerebras WSE.

To build this pipeline, we identify and address three domain challenges. (1) Accelerators can be programmed through Torch models; however, only a subset of Torch operators is supported, limiting cross-platform compatibility. To solve this, we build a queryable knowledge base of supported and unsupported operations on each platform, letting agents route around gaps within a bounded number of retries. (2) Most accelerators only support reduced-precision execution, a poor fit for scientific codes that expect double-precision accuracy. Thus, we characterize the stability of the generated kernels and use agent-proposed rewrites to recover precision. (3) Cross-platform evaluation requires orchestrating multiple servers. We use Globus Compute and Academy to abstract remote access and authentication across platforms, so agents can easily execute code remotely.

Across [N] kernels and [M] platforms, our pipeline produces a Pareto frontier trading off latency and numerical precision — spanning multiple equivalent Torch formulations, numerical formats (fp16, fp32, …), and error-compensation strategies (e.g., TwoSum accumulation) — recovering [X%] of double-precision accuracy at [Y%] of the latency of a CPU/GPU baseline, with [Z%] fewer manual engineering interventions than porting by hand.

Index Terms—Agentic AI, HPC, heterogeneous computing, accelerator, GPU, dataflow

I. Introduction

As AI workloads demand faster acceleration, the hardware accelerator landscape is becoming increasingly heterogeneous. Graphics Processing Units (GPUs) remain the dominant solution, while Domain Specific Accelerators (DSAs) are emerging as a compelling alternative [].

As discussed in Section II-A, hardware accelerators leverage techniques such as SIMT and dataflow parallelism to run parallel applications orders of magnitude faster or more efficiently than CPUs. Realizing those gains, however, is far from trivial. Each accelerator exposes a unique programming model, and designing architecture-aware algorithms requires domain expertise that most scientists do not have. While these researchers can typically write functionally correct code in their own fields (e.g., physics, chemistry), they often lack the specialized knowledge that hardware acceleration demands, including a deep understanding of algorithms, accelerator hardware, and how the two interact. Our pipeline helps scientists navigate these trade-offs by automating the generation and evaluation of scientific code for GPUs and other domain-specific accelerators.

To achieve this goal, we present a multi-agent framework that, starting from a reference implementation, synthesizes one or more equivalent Torch [1] models, verifies them against a high-precision oracle, and benchmarks them across platforms and numerical formats. For each kernel, the framework produces a performance–accuracy Pareto frontier spanning multiple machines, numerical formats, and implementations.

Using Torch provides a portable representation of the kernel that abstracts away low-level implementation details. The generated Torch model can be validated against the reference code on a CPU and then ported to any accelerator with a Torch-compatible SDK. Torch models also port easily across numerical formats. CPUs typically support 32- and 64-bit floating-point computation, while both GPUs and dataflow machines provide dedicated 16-bit floating-point units. On GPUs, 16-bit precision delivers significant speedups while a 32/64-bit fallback remains available; on most DSAs, 16-bit is the only option. Adapting the algorithm's numerical format is therefore essential to the pipeline's success.

Overall, the contributions of our work are:

- A multi-agent plan–translate–verify–benchmark loop that maps a reference kernel onto a {backend × precision × compensation} grid and returns non-dominated latency–accuracy points, with correctness gated by an fp64 oracle.
- Agent-discovered error-compensation variants .
- A machine-portable harness in which the agents' entire view of the world is a small workspace-confined tool surface, so the same agent code drives a local node, a remote GPU endpoint, and a dataflow accelerator without modification, and every measurement carries the machine that produced it.

Identify applicable funding agency here. If none, delete this. TODO

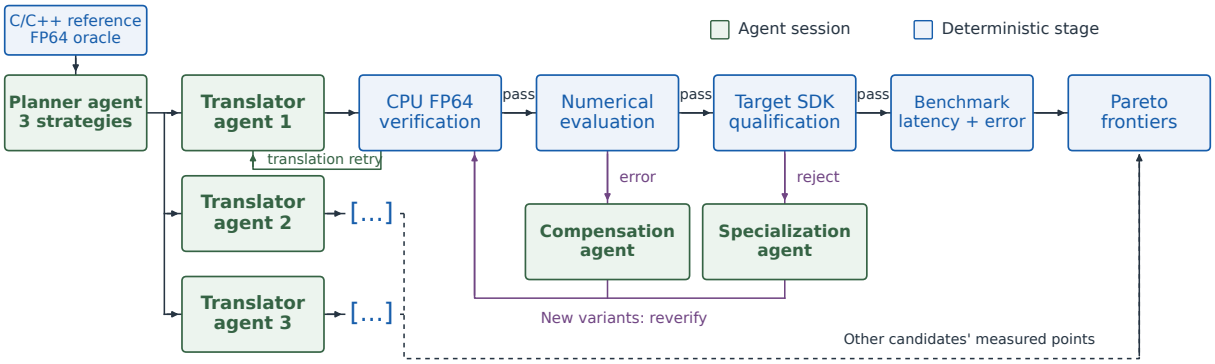


Fig. 1. Agentic pipeline overview.

II. Background

A. ML Accelerators and Portability

GPUs dominate machine learning and scientific computing because they combine high throughput with a mature and relatively general programming model. They execute large numbers of parallel threads over a hierarchical memory system spanning on-chip shared memory and caches, as well as off-chip high-bandwidth memory [2], [3]. Spatial and dataflow accelerators, by contrast, distribute computation across processing elements connected by on-chip networks, emphasizing static scheduling, data locality, and distributed storage [4]–[7].

The dataflow structure can reduce control and data-movement overheads relative to GPUs. This is particularly advantageous for low-batch-size ML inference, where dataflow architectures are known to excel [].

However, the same specialization that makes these accelerators attractive also limits their generality. Most ML accelerators are programmed primarily through tensor frameworks such as PyTorch rather than through general-purpose, low-level programming environments. This programmability gap has limited their applicability outside the ML domain. Nevertheless, prior work has demonstrated that the benefits of dataflow hardware extend beyond machine learning, with stencil computations providing a prominent example [8]–[10].

In this work, we demonstrate how agents can translate scientific code into tensor operations, enabling scientific applications to be ported to ML accelerators.

B. Reduced Precision and Scientific Accuracy

Scientific applications are commonly developed and validated using FP64 arithmetic, whereas machine learning applications typically rely on lower-precision formats such as FP16 and BF16. This distinction is also reflected in accelerator architectures. While GPUs generally retain FP32 and FP64 units to support scientific computation, AI accelerators primarily target 16-bit arithmetic. FP16's smaller exponent range increases the risk of overflow and underflow, whereas BF16's shorter significand increases rounding error and loss of significance [11].

A formally correct translation of a kernel into tensor operations can still produce incorrect results because of floating-point error. For this reason, our work separates semantic correctness from numerical correctness. We evaluate semantic correctness through high-precision testing and assess low-precision numerical correctness independently. Section IV-A characterizes the resulting failure modes and the algorithmic repairs used by the system to address them.

III. The Agentic Pipeline

Our pipeline starts from a scientific reference implementation in C or C++ and generates equivalent Torch implementations that are simultaneously semantically valid, numerically acceptable, and executable on heterogeneous ML accelerators. Rather than producing a single translation, the system generates and refines multiple candidate formulations, evaluating them across numerical formats and accelerator backends.

A. Kernels as Torch Models

Compared with approaches that generate accelerator-specific code directly [12], we express accelerator kernels in Torch. The main advantage is that Torch provides a common, executable representation across the stages of our workflow. The same kernel can first be validated against the C reference on CPU in FP64, then evaluated and repaired at reduced precision, and finally compiled for the target accelerator. This progressively separates semantic, numerical, and backend failures before asking an agent to address them.

Torch also spans a broad range of hardware, including CPUs, GPUs, and AI accelerators. Its explicit numerical formats make moving between FP64, FP32, BF16, and FP16 particularly convenient. This is important for AI accelerators, where reduced precision is often required.

B. Translating Kernels to Torch

The first step translates the reference C code into a Torch model. A planner agent proposes alternative

tensor formulations (three in our experiments), which are independently implemented by a translator agent.

Each candidate is executed on CPU in FP64 and compared against the original FP64 C implementation. A deterministic verifier checks that the implementation executes correctly and that its inputs and outputs match the reference within the numerical tolerances defined in Section V. Candidates that fail verification are returned to the translator with the observed errors for up to two correction rounds.

A candidate that passes this validation is considered a semantically valid translation and proceeds to reduced-precision evaluation and accelerator porting.

C. Numerical Evaluation

As discussed in Section II-B, AI-oriented accelerators typically benefit from reduced-precision formats, such as FP16. However, a CPU-validated FP64 kernel cannot be assumed to remain correct in reduced precision: overflow, underflow, and other floating-point errors are exacerbated by a different data type, leading to incorrect or diverging resulHots. Each floating-point format therefore requires independent evaluation and, if necessary, repair.

To study numerical stability, we evaluate each CPU-validated candidate in FP64, FP32, BF16, and FP16, casting its floating-point tensors to the requested precision and checking compliance with the precision policy. We compare its outputs against the C reference using the worst-output relative L_2 error, e. For each precision, we classify the candidate as stable ($e \leq 0.01$), concerning ($0.01 < e < 0.1$), or catastrophic ($e \geq 0.1$). These numerical classifications apply only after the required input, reference, and precision-policy checks pass; failures of these checks are reported separately. Following numerical repair, an improvement is classified as a full recovery if it meets the stability threshold, or a partial improvement if its error decreases but remains above that threshold.

D. Error-Compensation Agents

When a reduced-precision candidate fails the verification stage, the pipeline does not discard it but attempts to recover its accuracy with a specialized agent. The agent's goal is to recover accuracy while retaining as much of the reduced-precision performance as possible.

The agent starts from a copy of the unrepaired reduced-precision model and draws from a family of algorithm-level techniques that counteract the errors introduced by reduced precision without reverting to higher-precision hardware. Some examples are described in the remainder of this section. The repaired variant re-enters the verification and benchmarking loop as an additional candidate. The repair is considered successful when the candidate's output reaches the accuracy goal, which is 1% relative error in our evaluation. As in the translation step, the repair is declared failed after two unsuccessful correction rounds.

During repair, the agent is forbidden from using higher-precision tensor arithmetic, including widening the computation and casting the result back down. Accuracy must be recovered through algebraic reformulation rather than upcasting. Section IV-A discusses those techniques in greater detail.

E. Accelerator Specialization

The final subtask is the specialization of the kernel for the target accelerator. As discussed in Section IV-B, a valid Torch model may still be incompatible with the target hardware's SDK. This happens because Torch development typically outpaces that of the MLIR-based stacks behind domain-specific accelerators, whose frontends often implement only a subset of Torch operators.

To address this, a dedicated specialization agent adapts the kernels to work around these compiler limitations. Some limitations are trivially solved (e.g., $\log_2(x)$ is unsupported, while $\log(x)$ is), while others are unrecoverable.

To guide the specialization agent, we provide a dedicated MCP resource that acts as queryable documentation of the supported and unsupported functions on the target. The compiler limitations of each SDK are often undocumented, and without this resource the agent proceeds by trial and error, requiring an excessive number of steps. [...]

Overall, the pipeline proposes several Torch models that are first evaluated in double precision on the CPU, minimizing the chance of failure due to numerical and compiler errors. They are then tested in different numerical formats, which exposes the error stemming from the choice of data type. Wherever possible, this error is limited through dedicated rewriting. The model produced by the specialization agent is guaranteed to be algorithmically and numerically correct at the target precision. This design minimizes the number of kernels that must be compiled for the accelerator, one of the most costly steps of the pipeline. Moreover, starting from several versions of the algorithm increases the chance that at least one completes all the steps required for acceleration.

F. Performance Evaluation

Once all candidates have been validated as algorithmically and numerically correct on all platforms, we collect performance measurements to identify the best combination of algorithm, data type, and accelerator. Each measurement records the hardware platform (e.g., NVIDIA A100 or Groq LPU), the precision (e.g., FP16), the latency, and the accuracy on the evaluated workload. For latency, we only consider the forward-pass time, excluding model and input construction, staging, compilation, and orchestration.

The final output of the pipeline is a cloud of data points capturing the accuracy and latency of each generated variant. This point cloud identifies the Pareto front both per accelerator and across accelerators, showing the best attainable latency for a given target accuracy.

A candidate is Pareto-optimal if no other candidate is at least as fast and at least as accurate, and strictly better in one of the two. This point cloud identifies the Pareto front both per accelerator and across accelerators, showing the best attainable latency for a given target accuracy.

IV. Challenges

This section discusses the challenges that shaped the design of our agentic pipeline and the strategies that we put in place to overcome them.

A. Numerical Errors

As discussed in Section II-B, semantic correctness does not imply accuracy at a target precision. In practice, failures cluster around long reductions, recurrent updates, cancellation, ill-conditioning, and unstable function evaluation. The compensation agent first locates the earliest error-amplifying operation and classifies the mechanism as range loss, significance loss, cancellation, ill-conditioning, or function-evaluation error. It then maps the diagnosis to a repair catalogue including pairwise or compensated summation, double-word arithmetic, power-of-two scaling, centering, residual carry, and stable reformulation. Each technique has limits: scaling must protect intermediate values rather than only inputs, and compensation state stored in reduced precision can itself lose information. As required by Section III-D, all repairs remain in the target format and the verifier rejects upcasts.

We illustrate this process with a correlation kernel from the GPT 5.6 Sol runs. Its FP64 implementation is valid, but FP16 returns a non-finite output because reductions used to compute column means and standard deviations exceed the FP16 range. The agent diagnoses range loss and scales the data before the reductions. Because correlation is invariant to this scaling, the factor cancels from the result when the normalization threshold and fallback standard deviation are scaled consistently. The repaired kernel reaches a relative L_2 error of 5.07×10^{-4}.

B. Torch Portability Limitations

Despite the common Torch representation described in Section II-A, many CPU-valid kernels failed to compile on accelerator frontends. Trial-and-error discovery is costly because unsupported operators can be difficult to isolate inside a full model and each target compilation is expensive. We therefore characterize frontend coverage once and expose the results to the specialization agent through a queryable compatibility resource.

Starting from the Torch-MLIR operator list, we isolate 356 runnable forward tensor operators, wrap each in a minimal Torch model, and compile it on every accelerator. Figure 2 reports FP16 coverage by operator family. Graphcore compiles 261 operators (73%), Cerebras 234 (66%), and Groq 200 (56%); only 154 (43%) compile on all three. Thus, nominal support for a common Torch frontend still leaves substantial target-specific portability gaps.

Family	Total	Graphcore	Cerebras	Groq
Total	**356**	**261**	**234**	**200**
Arithmetic	71	55	55	53
Transcendentals	38	32	30	17
Linear algebra	27	18	14	14
Complex / FFT	3	1	1	0
Convolution	11	8	0	5
Pooling	18	6	9	5
Activations	26	24	20	21
Normalization	8	1	0	5
Reductions	40	32	29	31
Indexing	17	9	11	7
Shape / layout	76	63	57	39
Losses	8	7	6	3
Sampling	7	5	2	0
Quantization	6	0	0	0

Fig. 2. Compilation coverage of the 356 runnable operators, overall and by operator family. Numbers above the bars give the number of compiled operators, and bar lengths show their fraction of the family total. Results use PopTorch 3.3.0 (Graphcore), Cerebras PyTorch 2.10.0, and GroqFlow 4.3.1.

For each operator, the resource records canonical inputs, compiler outcome, and diagnostics for every target. Before recompiling a rejected kernel, the specialization agent queries uncertain operators and searches for supported alternatives. This converts repeated compiler failures into a bounded search over known backend capabilities; the knowledge-base ablation later evaluates its value explicitly.

C. Remote Execution

Cross-platform evaluation requires interacting with accelerators behind different login, scheduling, and software environments while preserving state across agent iterations. We use Globus Compute [13] for remote execution and Academy [14] for persistent agents at each endpoint, providing a common interface through which the same workflow operates across local, GPU, and dataflow systems.

V. Experimental Setup

This section describes the experiments that we used to evaluate the pipeline.

A. Kernels

We evaluate 33 kernels: 24 selected from PolyBench/C 4.2.1 [15] and nine scientific kernels. Table I groups them by domain and lists their computations and workload sizes. Together, they cover dense and sparse linear algebra, grid methods, dynamic programming, physical simulation, transforms, and signal processing.

TABLE I
List of evaluated kernels: 24 from PolyBench/C (above the divider) and nine scientific additions.

Domain	Kernels	Computation	MINI	LARGE
Statistics	correlation, covariance	Correlations, covariances	32×28	1400×1200
Matrix and tensor algebra	gemm 3mm atax, bicg doitgen gemver, mvt gesummv syr2k, trmm	Matrix products, rank updates, contractions	$20 \times 25 \times 30$ 16–24 (five dimensions) 38×42 $8 \times 10 \times 12$ 40×40 30×30 20×30	$1000 \times 1100 \times 1200$ 800–1200 (five dimensions) 1900×2100 $140 \times 150 \times 160$ 2000×2000 1300×1300 1000×1200
Linear algebra	cholesky, lu durbin gramschmidt	Factorizations, Toeplitz solve	40×40 vector of 40 20×30	2000×2000 vector of 2000 1000×1200
Image filtering	deriche	Recursive smoothing	64×64 image	4096×2160 image
Graphs and RNA	floyd-warshall nussinov	Shortest paths, base pairing (DP)	60 nodes 60 bases	2800 nodes 2500 bases
PDEs and grids	adi fdtd-2d heat-3d jacobi-2d seidel-2d	Diffusion, electromagnetism, relaxation	20 steps, 20×20 20 steps, 20×30 20 steps, $10 \times 10 \times 10$ 20 steps, 30×30 20 steps, 40×40	500 steps, 1000×1000 500 steps, 1000×1200 500 steps, $120 \times 120 \times 120$ 500 steps, 1300×1300 500 steps, 2000×2000
Spectral analysis	direct-dft	Direct Fourier sums	32 samples	4096 samples
Gravitation	softened-nbody	Pairwise accelerations	32 particles	2048 particles
Nonlinear dynamics	lorenz-rk4	Lorenz system, RK4	32 trajectories, 20 steps	1024 trajectories, 200 steps
Sparse systems	poisson-cg	10 CG iterations, 1D Poisson	32 unknowns	16384 unknowns
Information theory	mutual-information	Mutual information	256×256	256×256
Image compression	jpeg-dct wavelet-compression low-rank-compression	Block DCT, 8×8 Haar, 3 levels Rank 4, 8 iterations	32×32 image	1024×1024 image
Signal processing	iir-filter-bank	Four 2nd-order IIR filters	32 samples	16384 samples per filter

All workloads use deterministic inputs, and accuracy is measured against the output of the corresponding C reference in FP64. We use two sizes: MINI for CPU translation checks, and LARGE for accuracy validation, timing, and numerical repair. For the three image-compression kernels, accuracy measures agreement with the reference compression algorithm, not reconstruction error with respect to the original image.

B. Hardware Platforms

We evaluate one GPU and three dataflow accelerators, representing substantially different execution and memory architectures.

The NVIDIA A100 is our GPU reference. We use the 40-GB PCIe version, with 6,912 CUDA cores, 432 Tensor Cores, 40 GB of HBM2 at 1.555 TB/s, and 312 TFLOP/s peak dense FP16 throughput [16]. It is programmed through PyTorch on CUDA.

The Groq LPU uses 320 statically scheduled lanes with 230 MB of on-chip SRAM and no attached DRAM, providing 80 TB/s of memory bandwidth and 188 TFLOP/s peak FP16 throughput [17]. Kernels are compiled through GroqFlow and GroqWare.

The Cerebras CS-3 uses the wafer-scale WSE-3, with approximately 900,000 Processing Elements (PEs) and 44 GB of distributed on-chip SRAM providing 21 PB/s of bandwidth [18], [19]. Its peak sparse FP16 throughput is 125,000 TFLOP/s. Kernels are compiled through Cerebras PyTorch.

The Graphcore Bow IPU contains 1,472 independent tiles, each combining compute with local SRAM and communicating through bulk-synchronous phases [20], [21]. It provides 900 MB of on-chip SRAM, approximately 65 TB/s of bandwidth, and about 350 TFLOP/s peak FP16 throughput. Kernels are compiled through PopTorch and Poplar.

C. Metrics

We evaluate implementations in terms of latency and numerical accuracy.

The completed Groq and Graphcore portability records report median forward-pass latency over 20 samples after 3 warmups, using the vendor benchmark or device cycle

counter for one device-resident invocation per sample. A100 observations retain their per-run timing protocol and sample counts. SDK host wall time and unavailable native device latency are reported separately rather than substituted for device timing.

Accuracy is measured against the FP64 C reference as the worst relative L_2 error across outputs, $\max_j \|c_j - o_j\|_2 / \|o_j\|_2$. We use 1% error as the threshold for triggering numerical repair; it is a pipeline routing criterion rather than a general definition of scientific correctness. Non-finite or invalid outputs are always considered failures.

VI. Results

A. C-to-Torch Translation

We first assess how many kernels have at least one valid C-to-Torch candidate. As shown in Table II, Luna covers 32/33 kernels (97.0%) in the MINI suite, while Terra and SOL cover all 33 kernels. Deriche is the only kernel without a validated Luna implementation.

The selected runs contain 94 generated candidates for Luna, 98 for Terra, and 99 for SOL, with 92, 96, and 98 final valid implementations, respectively. We count distinct candidate strategy slots with recorded outcomes; revisions of the same candidate are counted once. Of the 99 planned slots per model, five Luna slots and one Terra slot have no recorded outcome and are excluded from the generated counts. Across the three models, the final inventory contains 286 valid implementations.

TABLE II
CPU FP64 translation coverage and token usage on the 33-kernel MINI suite. A covered kernel has at least one valid candidate. Generated counts distinct candidate slots with recorded outcomes; valid counts final validated implementations. Input/output token totals are in millions across the 33 selected runs per model. Valid GPT-OSS versions include four separately validated replacements; usage covers the original selected runs.

Model	Kernels covered	Generated	Valid	Input (M)	Output (M)
Luna	32/33	94	92	5.39	0.69
Terra	33/33	98	96	5.20	0.69
SOL	33/33	99	98	5.91	0.90
GPT-OSS 120B	29/33	95	68	28.00	1.11

We next examine how many verification attempts are needed to obtain an accepted candidate (Table III). Here, pass@k denotes cumulative recorded CPU FP64 gate acceptance within k successive verifier submissions of the same candidate: pass@1 is the first submission, pass@2 allows one correction after rejection, and pass@3 allows two corrections. These are rates for a sequential correction process, rather than estimates based on independent samples. Each model's generated candidate count is the denominator.

Luna, Terra, and SOL pass the first verification on 82, 84, and 92 candidates, respectively. One correction recovers an additional 7, 8, and 5 candidates; a second correction recovers another 3, 4, and 1. Thus, the first correction accounts for 20 of the 28 recoveries across the three models. Two Luna candidates, two Terra candidates, and one SOL candidate remain rejected after the correction budget is exhausted.

TABLE III
Recorded CPU FP64 gate acceptance on the 33-kernel MINI suite. Here, pass@k counts candidates accepted within k successive verification attempts, including revisions after rejection. Fractions use recorded candidate outcomes; planned slots without an outcome are excluded. These original gate counts precede the retrospective source audit and separate corrections.

Model	pass@1	pass@2	pass@3
Luna	82/94	89/94	92/94
Terra	84/98	92/98	96/98
SOL	92/99	97/99	98/99
GPT-OSS 120B	48/95	64/95	71/95

These results describe validated implementations at the configured MINI workload. Validation checks agreement with the C FP64 reference and input initialization; it does not establish correctness for all inputs or shapes, or certify GPU and reduced-precision execution. GPT-OSS records 95 candidate outcomes across the same 33 kernels: 48 pass the first gate, 16 more pass after one correction, and seven more after two. Of these 71 originally accepted sources, the retrospective audit retains 64. Seven separate correction trials accept four replacements, yielding 68 usable implementations across 29 kernels. The remaining original outcomes are 22 rejections and two crashes. Four planned slots have no recorded outcome. Original attempt counts and later source corrections are reported separately.

a) Recorded generation effort: Table II separates aggregate input and output token usage across the 33 selected kernel runs per model. We sum recorded usage for the planner and all candidate agents, including unsuccessful candidates and in-run corrections. Input tokens account for 86.8–96.2% of aggregate usage. These measurements cover usage within the selected run records; work outside those records is excluded. Token counts reflect reported usage and are not monetary costs.

B. Numerical Repair

We next evaluate whether numerical repair can recover accuracy without widening tensor arithmetic. Each target is a kernel, candidate strategy, and precision combination, evaluated at LARGE size on the NVIDIA A100. All models receive the same Terra baseline for a given target, with common C-initialized inputs. Full recovery requires a worst-output relative L_2 error of at most 1%, together with the semantic and precision checks described in Section III-D. An outcome labeled improved only lowers the error but remains above the accuracy threshold; we also call this a partial improvement. These two categories

are disjoint. Any improvement denotes their union and includes fully recovered targets.

The recovery counts in this section refer to the development fixture. Section VII-C reports independent-input validation for every eligible FP16 full recovery.

Table IV reports the eligible inventory for 60 common targets across 21 kernels: 33 FP16 targets and 27 BF16 targets. Luna, Terra, and SOL fully recover 10 (16.7%), 15 (25.0%), and 19 (31.7%) targets, covering seven, nine, and 11 kernels, respectively. Luna improves another 14 targets without reaching the threshold; Terra and SOL each improve another 12. Any improvement therefore covers 24/60 Luna targets, 27/60 Terra targets, and 31/60 SOL targets, including full recoveries. The FP16/BF16 full-recovery counts are 4/33 and 6/27 for Luna, 7/33 and 8/27 for Terra, and 9/33 and 10/27 for SOL. These are repairs of initially inaccurate implementations; implementations already within the threshold are outside this repair cohort. GPT-OSS completes the same 60-target inventory with zero accepted full recoveries and zero partial improvements (0/33 FP16 and 0/27 BF16 recoveries). Its outcomes comprise 56 rejections and four crashes; 13 are retained historical outcomes and 47 are completed current-protocol continuations. Its finite-baseline error reduction is therefore 1.00×.

TABLE IV
Accuracy repair on 60 common LARGE targets across 21 kernels. Fully recovered means an eligible repair reaches ≤ 1% error; improved only means an eligible repair lowers error but remains above 1%. These columns are disjoint; their sum counts all improved targets. Recovered kernels have at least one full recovery. Error reduction is the geometric mean over the same 29 finite-baseline targets, including both recovery categories and 1× for unsuccessful or held repairs. It is not a recovery rate. Input/output tokens are in millions. GPT-OSS token totals are recorded lower bounds; timed-out requests can have missing usage.

Model	Recovered kernels	Fully recovered	Improved only	Error reduction	Input (M)	Output (M)
Luna	7/21	10	14	2.31×	6.05	1.08
Terra	9/21	15	12	3.58×	6.06	0.87
SOL	11/21	19	12	4.98×	5.27	0.81
GPT-OSS 120B	0/21	0	0	1.00×	20.90	1.54

Table V distinguishes the two disjoint outcome categories for every kernel, including targets with non-finite baseline outputs. Each pair lists fully recovered targets first and improved-only targets second. For example, SOL recovers one direct DFT target and improves another without recovery; Terra improves two DFT targets without recovery.

a) Magnitude of accuracy improvement: Recovery counts indicate whether a repair crosses the accuracy threshold; we also measure how much it reduces error. For target t and model m, let $e_{0,t}$ be the shared baseline error and $e_{m,t}$ the error retained after repair. We retain an eligible repair only when it improves the baseline; rejected, failed, or held repairs retain the baseline and therefore contribute 1×. Over the common set F of finite, positive baseline errors, the geometric mean reduction is

$$G_m = \exp\left(\frac{1}{|F|}\sum_{t\in F}\log\frac{e_{0,t}}{e_{m,t}}\right).$$

The same 29 targets across 13 kernels enter all three model means: Luna achieves 2.31× error reduction, Terra 3.58×, and SOL 4.98×. The other 31 targets have non-finite baseline outputs and are excluded from fold reductions; their repair outcomes remain in the counts in Table V. The mean measures the magnitude of error reduction across both accepted outcome categories and unsuccessful repairs, rather than the fraction of targets fully recovered.

TABLE V
Per-kernel repair outcomes on all N common targets. Each model cell gives two disjoint counts: fully recovered / improved only. Fully recovered reaches ≤ 1% error; improved only lowers error but remains above 1%. Their sum is any improvement. Rejected, failed, or held proposals count in neither category. Non-finite baseline cases remain included here. All four models use the same targets.

		Fully recovered / improved only			
Kernel	N	Luna	Terra	SOL	OSS
3mm	3	0 / 0	0 / 0	0 / 0	0 / 0
adi	1	0 / 0	0 / 0	1 / 0	0 / 0
atax	3	0 / 0	0 / 0	0 / 0	0 / 0
bicg	1	1 / 0	1 / 0	1 / 0	0 / 0
cholesky	4	0 / 0	0 / 0	0 / 0	0 / 0
correlation	2	1 / 1	2 / 0	2 / 0	0 / 0
covariance	2	0 / 0	0 / 0	0 / 0	0 / 0
deriche	1	0 / 1	0 / 1	0 / 1	0 / 0
direct-dft	4	0 / 1	0 / 2	1 / 1	0 / 0
gemver	3	0 / 0	0 / 0	0 / 0	0 / 0
gesummv	1	1 / 0	1 / 0	1 / 0	0 / 0
jacobi-2d	2	1 / 1	2 / 0	2 / 0	0 / 0
jpeg-dct	6	0 / 1	0 / 1	0 / 1	0 / 0
lorenz-rk4	2	0 / 2	1 / 1	2 / 0	0 / 0
low-rank-compression	3	0 / 1	1 / 1	2 / 1	0 / 0
lu	6	0 / 0	0 / 0	0 / 0	0 / 0
mutual-information	3	3 / 0	3 / 0	3 / 0	0 / 0
poisson-cg	6	2 / 1	3 / 0	3 / 2	0 / 0
softened-nbody	4	0 / 3	0 / 4	0 / 4	0 / 0
trmm	1	1 / 0	1 / 0	1 / 0	0 / 0
wavelet-compression	2	0 / 2	0 / 2	0 / 2	0 / 0
Total	60	10 / 14	15 / 12	19 / 12	0 / 0

Table VI reports geometric mean error reduction separately from the outcome counts in Table V. Its factors use only finite-baseline targets, so a dash does not imply that no target was recovered. Gains vary substantially: Terra achieves 175.93× on Jacobi-2D, versus SOL's 52.47×, while SOL achieves 144.51× on direct DFT, versus Terra's 38.34×. The overall reduction weights individual targets equally; it is not an unweighted mean of the kernel rows.

To examine the contribution of feedback, Table VII isolates completed repair invocations with recorded verification histories: 36 Luna cases, 37 Terra cases, and 36 SOL cases. Both column groups are cumulative: fully recovered by attempt k counts threshold-crossing recoveries within k submissions, while any improvement by attempt k

TABLE VI

Geometric mean error reduction (×) by kernel. This measures the magnitude of error reduction, not how many targets reach 1% error. Both fully recovered and improved-only repairs contribute; unsuccessful or held repairs retain the baseline (1×). n/N is the finite-baseline target count over all repair targets. A dash indicates no finite baseline; recovery counts are reported separately in Table V. The overall row weights the 29 finite-baseline targets equally.

Kernel	n/N	Luna	Terra	SOL	GPT-OSS 120B
3mm	0/3	—	—	—	—
adi	0/1	—	—	—	—
atax	0/3	—	—	—	—
bicg	1/1	154.99	154.99	154.99	1.00
cholesky	0/4	—	—	—	—
correlation	0/2	—	—	—	—
covariance	0/2	—	—	—	—
deriche	1/1	1.01	2.91	2.91	1.00
direct-dft	2/4	4.01	38.34	144.51	1.00
gemver	0/3	—	—	—	—
gesummv	1/1	6.07	7.79	6.07	1.00
jacobi-2d	2/2	35.23	175.93	52.47	1.00
jpeg-dct	5/6	1.18	1.18	1.18	1.00
lorenz-rk4	2/2	1.00	4.45	10.81	1.00
low-rank-compression	0/3	—	—	—	—
lu	2/6	1.00	1.00	1.00	1.00
mutual-information	3/3	2.42	2.79	5.90	1.00
poisson-cg	3/6	1.00	1.00	6.33	1.00
softened-nbody	4/4	1.02	1.02	1.02	1.00
trmm	1/1	13.07	13.06	13.03	1.00
wavelet-compression	2/2	1.98	2.26	2.26	1.00
Overall	29/60	2.31	3.58	4.98	1.00

TABLE VII

Cumulative outcomes within 1, 2, or 3 verification attempts of the selected repair invocation. Fully recovered reaches ≤ 1% error. Any improvement includes full recoveries and improvements that remain above 1%; full recoveries are therefore included in both groups. Each later column includes earlier successes. Held proposals count in neither group. Denominators are 36 Luna, 37 Terra, 36 SOL, and 47 GPT-OSS invocations, a subset of the inventory in Table IV. Crashes remain in the invocation denominator.

	Fully recovered by attempt			Any improvement by attempt		
Model	1	2	3	1	2	3
Luna	0/36	0/36	0/36	1/36	3/36	4/36
Terra	1/37	2/37	2/37	4/37	6/37	6/37
SOL	4/36	5/36	5/36	6/36	8/36	8/36
GPT-OSS 120B	0/47	0/47	0/47	0/47	0/47	0/47

additionally includes improvements that remain above the threshold. SOL fully recovers four cases on the first attempt and one more after a correction; Terra recovers one on the first attempt and one more after a correction. Luna adds no full recoveries in this invocation subset. Including improved-only outcomes gives final cumulative totals of four Luna cases, six Terra cases, and eight SOL cases. Luna's second correction adds one improved-only outcome. Retained results, including Luna's 10 full recoveries, contribute to the inventory in Table IV but are not assigned a reconstructed retry history. GPT-OSS completes 47 selected invocations with no accepted improvement at any of the three attempt limits. These invocation-level rates therefore use a different denominator from the 60-target inventory and do not measure the total number of earlier attempts.

b) Numerical transformations: Successful repairs address both range loss and significance loss. For low-rank compression in FP16, SOL rescales vectors before squared-norm reductions and applies a common scale to the residual and signal energy sums. The scaling cancels algebraically, while preventing overflow in intermediate reductions. Two candidate strategies reach relative L_2 errors of 2.02×10^{-3} and 1.15×10^{-3}. For Jacobi-2D c2 in FP16, Terra instead rewrites the five-point average as an update from local differences, reducing recurrent rounding drift. Its error falls from 1.25×10^{-1} to 2.38×10^{-4}. Both examples preserve the validated FP32/FP64 path.

Direct DFT in BF16 illustrates why improvement alone is insufficient. On the same c3 target, whose baseline error is 1.60, phase-reduction reformulations yield an error of 7.16×10^{-3} for SOL and 9.93×10^{-2} for Terra. Both improve the baseline, but only SOL crosses the 1% threshold. Across the matched target set, the 14 Luna and 12 Terra/SOL improved-only outcomes therefore remain separate from full recoveries.

c) Effort and failures: Table IV includes recorded input and output tokens for the selected target set, counting unsuccessful proposals and correction work as well as successful repairs. Existing usage is counted once when a checkpoint is continued. These totals exclude translation, baseline measurement, and subsequent semantic follow-ups; they are not complete end-to-end pipeline costs. In the separate corrected-common-Terra five-arm follow-up, SOL and Terra partially improve Wavelet FP16 (errors 0.07729 and 0.09806 from 2.02161), while Luna Wavelet and both GPT-OSS arms are rejected. None is a full recovery; these follow-ups are excluded from the matched primary table. Luna has 35 rejected targets and one held partial observation whose measured source hash differs from the saved implementation; that observation is excluded from accepted improvement counts, while its usage is retained. Terra has 31 rejected targets and two execution failures. SOL has 22 rejected targets, two execution failures, and five held JPEG proposals in this primary inventory. GPT-OSS records 20,897,510 input and 1,535,271 output tokens for the primary inventory, counting historical usage once and continuation usage separately. These are lower bounds on returned usage: timed-out requests can incur unreported tokens. The two held targets in its original 49-entry continuation worklist are outside this eligible 60-target inventory. Rejection can occur before a numerical result, so these outcomes must not all be interpreted as failures to reduce error.

C. Accelerator Portability

We next port selected, source-bound A100 FP16 implementations to the three dataflow accelerators. Groq's frozen worklist contains 34 source versions across 14 applications; Graphcore and Cerebras contain 36 across 15 applications, including two later diversity additions. The worklists include distinct versions with the same candidate labels. These counts describe configured-workload numerical passes under the backend's compiler precision contract; they do not establish arbitrary-input correctness or a separately audited internal accumulator precision.

TABLE VIII
Earlier snapshot of FP16 destination numerical results on LARGE workloads using the accepted compiler precision contract. A pass requires verified C inputs, finite complete outputs, and worst-output relative $L_2 \leq 1\%$. Cases identify source versions; applications have at least one pass. Pending includes interrupted infrastructure. Cerebras was in progress at this snapshot; Table IX uses later timing evidence from its completed portfolio.

Target	Passing cases	Passing applications	Unqualified terminal	Pending
Groq	16/34	7/14	18	0
Graphcore	18/36	9/15	18	0
Cerebras	1/36	1/15	1	34

The earlier portability snapshot records all 34 Groq cases, with 16 numerical passes and 18 unqualified outcomes. Graphcore completes all 36 main cases with 17 numerical passes. Three deterministic runtime follow-ups reuse existing case IDs: BICG c1 adds one pass, while both IIR cases time out. The effective result is therefore 18 numerical passes and 18 unqualified cases over the same 36-case denominator; the 39 raw result records are retained. Cerebras was still in progress when the snapshot in Table VIII was assembled. The later timing audit below uses the completed CS-3 portfolio, with 11 numerical passes and 25 unqualified cases across the same 36-case worklist.

a) Specialization and measurement scope: Ports retain the accepted source algorithm and receive C-input parity, CPU semantic, immutable-input, and repeatability checks. Compatibility agents receive at most two correction rounds. Independent destination alternate-input validation remains a separate task: the A100 repair holdout is not a certificate for a compiled Groq, IPU, or WSE program. Unqualified ports include compiler, runtime, numerical, and resource failures; compilation alone is not counted as a numerical pass. Groq SDK invocation timing and Graphcore device timing have distinct scopes and are not pooled into one speedup statistic. Cerebras SDK host throughput is retained separately from native device latency, which remains unavailable in the passing records; compiler-derived estimates are reported separately below.

D. Transfer-Inclusive and Resident Execution

Table IX consolidates the available timing evidence for 21 source-version cases across 11 applications at LARGE size in FP16. The timing cohort is the union of cases with qualified observations or compiler estimates; it is not the denominator of the earlier portability snapshot. Measured call and resident series use 20 serialized invocations after three warmups, with setup excluded. The A100 active profiles contain 20 additional replays of the same captured graph; Lorenz c3 reuses qualified archived A100 call/resident measurements and has no active profile. Backend compatibility ports can differ, so case identity matches workload/source lineage rather than identical code across devices.

Transfer-inclusive calls and device-only execution evidence answer different questions. CUDA-event resident time retains scheduling gaps, whereas the profiled active union excludes idle intervals. Groq reports an exact-executable compiler schedule and the IPU estimate reconstructs active internal phases. CS-3 streaming intervals include modeled stalls; its ideal-residency columns instead sum annotated wafer-native cycles at the compiler's 750 MHz target clock, with and without the existing context-switch budget. They assume preloaded layouts and do not establish resident-placement feasibility or a new executable's latency. Five CS-3 cases also have uncosted host helpers. These distinctions, the IIR profiling anomaly, and timing differences between runs prevent interpreting arbitrary column ratios as measured architectural speedups or differences as isolated PCIe costs.

E. Latency–Accuracy Trade-offs

Placeholder: Compare measured latency and numerical error across validated implementations, precisions, repairs, and accelerators. Identify which choices contribute to each frontier and how the fastest valid implementation changes with the accuracy requirement.

a) Scope and measurement cost: Placeholder: State the reference implementation, accuracy metric, workload sizes, and timing protocol used in the comparisons. Report compilation and benchmarking costs separately from forward-pass latency, and identify missing target measurements.

VII. Ablation Studies

A. Knowledge Base Ablation

Can agents figure out Torch limitations without the KB?

B. Multi-Agent Pipeline Ablation

Can a single agent carry out the whole process without the multi-agent pipeline?

C. Generalization Beyond the Repair Fixture

The benefits of numerical repair extend beyond the development fixture, even when held-out errors remain above the 1% accuracy threshold. To measure this transfer, we compare all twenty FP16 implementations recorded as fully recovered in the primary inventory of Table IV with

TABLE IX

Transfer-inclusive timings and resident/active execution evidence for LARGE FP16 cases (all values in milliseconds). M: measured; P: profile-derived estimate; C: compiler-reported schedule estimate; D: derived native-cycle budget. Distinct timing scopes are retained rather than pooled into a common speedup.

		NVIDIA A100			Groq		Graphcore IPU		Cerebras CS-3			
Kernel	Case	I/O+ M	Resident M	Active P	I/O+ M	Schedule C	I/O+ M	Active P	Stream C	First C	Native D	Native+ctx D
BICG	c1	0.4921	0.099328	0.03638	1.1245	0.12061	2.613	0.11662	–	–	–	–
	c2	0.545	0.13875	0.10286	1.0585	0.055999	2.4901	0.016811	3.1334	4.6355	0.011681	0.18368
	c3	1.9801	1.5508	1.3474	1.1744	0.17106	2.5353	0.016215	3.1334	4.6355	0.011681	0.18368
MVT	c1	0.53198	0.10906	0.04384	1.1029	0.048283	2.5716	0.068494	3.2704	4.7629	0.014765[b]	0.30143[b]
	c2	0.54022	0.10291	0.03726	1.0897	0.048283	2.5536	0.068494	6.9289	6.9863	0.082731	0.42673
	c3	0.60674	0.16128	0.090139	1.1336	0.061336	2.538	0.026278	3.2704	4.7629	0.014765[b]	0.30143[b]
GESUMMV	c1	0.4368	0.098816	0.03554	0.89706	0.034023	2.0517	0.0070254	3.8044	3.8045	0.004116[b]	0.17612[b]
	c2	0.46472	0.10138	0.04742	0.89382	0.030992	2.0637	0.0077714	3.8044	3.8045	0.004116[b]	0.17612[b]
GEMM	c2	0.65502	0.14848	0.08684	1.9243	0.078151	5.5945	0.068494	–	–	–	–
	c3	0.67872	0.15667	0.1027	2.0752	0.078151	5.6157	0.068495	6.8837	6.8837	0.045416[b]	0.27475[b]
IIR filter bank	c1	202.17	202.1	237.41[a]	0.05143	0.014652	–	–	–	–	–	–
Lorenz RK4	c3	15.576	17.186	–	–	–	–	–	102.76	102.82	10.138	102.67
Low-rank	c1	2.9731	2.7448	2.3519	–	–	4.9961	0.62482	–	–	–	–
	c2	2.1075	1.877	1.5849	–	–	8.7854	0.41413	–	–	–	–
Poisson CG	c1	0.7391	0.6999	0.83097	0.08626	0.066423	0.35793	0.10574	–	–	–	–
DOITGEN	c1	0.73928	0.091648	0.03964	2.444	0.10227	20.983	0.019367	11.795	11.795	0.070929	0.1856
	c3	4.1502	3.5471	3.5872	2.3144	0.10227	20.918	0.019367	19.007	19.007	0.015199	0.12987
SYR2K	c1	0.74724	0.19558	0.15446	3.0047	0.10915	6.9695	0.11696	–	–	–	–
	c2	0.74486	0.19866	0.17588	3.0178	0.11848	6.7797	0.091851	–	–	–	–
TRMM	c1	0.51442	0.13978	0.087739	1.976	0.059546	10.378	0.14558	–	–	–	–
	c3	5.8306	8.8325	5.0293	1.9488	0.057678	–	–	–	–	–	–

Timing boundaries. I/O+ is a measured serialized host-to-host call, including input/output transfers and invocation, with setup excluded. A100/Groq use PCIe; Graphcore uses the deployed IPU fabric. A100 Resident is CUDA-event elapsed time for a captured graph with device-resident tensors (including HBM access); Active is the union of profiled kernel/device-memory intervals with idle gaps excluded. Groq Schedule is the successful executable's compiler cycle estimate. IPU Active reconstructs compute, internal exchange and internal synchronization phases.

CS-3 estimates. Stream is the predicted steady-state interval with modeled streaming stalls; First is the predicted first compiled iteration. Neither is a measured host-to-host call. Native sums the compiler's wafer-native operation cycles assuming preloaded data/layouts, resident code and serial operations; Native+ctx also retains the original context-switch cycle budget. Both D columns are conditional component budgets, not predictions from a newly compiled resident executable or proof of memory-placement feasibility.

[a] IIR profiling perturbs timing; its active estimate exceeds the same process's capture-disabled resident interval and is diagnostic only. [b] CS-3 has uncosted host helpers (MVT addition; GESUMMV/GEMM reduction/distribution), so the D columns omit required remapping costs. Independent runs and instrumentation can also make active estimates exceed archived elapsed medians; differences are not isolated transfer costs. – denotes unavailable qualified timing evidence, not zero latency. Case IDs and source hashes are recorded in the accompanying CSV.

their paired unpatched sources: four Luna, seven Terra, and nine SOL implementations across five kernels. Each receives twelve deterministic holdouts: four local perturbations at two magnitudes, two independent random inputs, half- and double-scale inputs, two structured fields, and two sparse interventions. Workload dimensions, iteration counts, and fixed parameters are unchanged. Sources, C contracts, and input policies are frozen; no repair uses holdout feedback.

Both versions receive identical inputs on the A100. Expected outputs come from the original C computation with inputs injected after initialization and dependent state rebuilt consistently. We report the repair loop's worst-component relative L_2 error in percent:

$$e = 100 \max_j \frac{\|\widehat{y}_j - y_{C,j}\|_2}{\max(\|y_{C,j}\|_2, 10^{-30})}, \tag{1}$$

where j indexes independently scored C outputs, including scalars. Finite minima, medians, and maxima exclude unavailable numerical errors; their counts accompany every summary in Table X. NaN/Inf outputs receive no arbitrary percentage. Repeated forwards and input/dtype audits accompany the measurements; all 260 unpatched FP64 checks, including canonical controls, satisfy the C semantic check.

Across 240 implementation–input holdouts, unpatched FP16 sources produce 196 NaN outputs and 44 finite outputs. Repair makes 187 of those previously non-finite observations finite, yielding 231 finite outputs, seven Inf outputs, and two NaN outputs. Source-bound replays confirm the non-finite values, including NaNs whose repeating locations triggered the original strict repeatability rejection. These changes demonstrate transferred improvements in numerical stability; finiteness alone does not establish accuracy.

Jacobi-2D provides a fully finite accuracy comparison. Unpatched errors span 4.266–12.605%, with median 12.482%; repaired errors span 0.0238–1.042%, with median

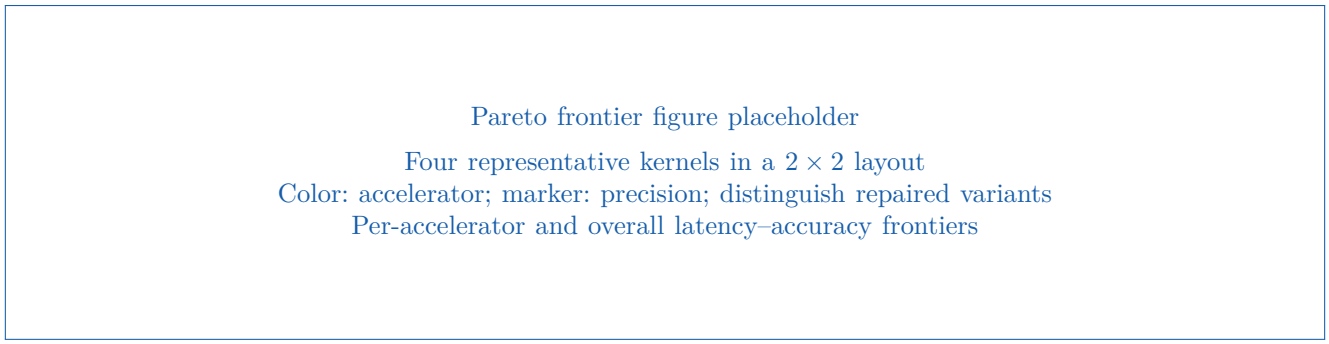


Fig. 3. Placeholder: Latency–accuracy trade-offs for four representative kernels. Show validated measured points, repaired variants, accuracy thresholds, and the non-dominated frontiers.

TABLE X
Paired holdout errors: twenty primary FP16 repairs and five diagnostics (†). Entries: finite min / median / max relative L_2 error (%) [unavailable out of 12]. NaN or n/a: no finite error; —: no source.

Target	Unpatched	Repaired: Luna	Repaired: Terra	Repaired: SOL
Poisson-CG c1	0.0935 / 0.0935 / 0.0935 [11]	0.0917 / 0.504 / 2.08 [1]	0.0919 / 0.801 / 5.33 [1]	0.0919 / 0.801 / 5.33 [1]
Poisson-CG c2	0.0935 / 0.0935 / 0.0935 [11]	0.0784 / 0.439 / 1.41 [1]	0.0688 / 0.248 / 1.05 [1]	0.0877 / 0.6 / 5.04 [1]
Poisson-CG c3	0.0935 / 0.0935 / 0.0935 [11]	—	0.0877 / 0.6 / 5.04 [1]	0.0877 / 0.6 / 5.04 [1]
Correlation c1	NaN [12]	0.0462 / 0.0472 / 0.117 [1]	0.0462 / 0.0477 / 99.6 [0]	0.0462 / 0.0474 / 0.117 [0]
Correlation c3	NaN [12]	—	0.0663 / 0.0794 / 0.126 [0]	0.0351 / 0.0374 / 0.119 [0]
Jacobi-2D c2	4.27 / 12.5 / 12.6 [0]	0.0477 / 0.184 / 1.03 [0]	0.0238 / 0.174 / 1.04 [0]	0.0477 / 0.184 / 1.03 [0]
Low-rank c1	NaN [12]	—	0.115 / 0.162 / 1.31 [0]	0.122 / 0.2 / 0.82 [0]
Low-rank c2	NaN [12]	—	—	0.115 / 0.162 / 1.31 [0]
ADI c2	NaN [12]	—	—	0.00781 / 2.79 / 27.3 [0]
DFT c3 (BF16)†	130 / 160 / 160 [0]	—	—	0.702 / 0.736 / 0.812 [0]
JPEG c1†	22.7 / 22.7 / 22.7 [11]	—	—	0.227 / 0.227 / 0.227 [11]
JPEG c2†	22.7 / 22.7 / 22.7 [11]	—	—	0.227 / 50.1 / 100 [10]
JPEG c3†	n/a [12]	—	—	0.227 / 50.1 / 100 [10]
Jacobi-2D c3 (held)†	4.23 / 12.5 / 12.6 [0]	—	—	0.0238 / 0.174 / 1.04 [0]

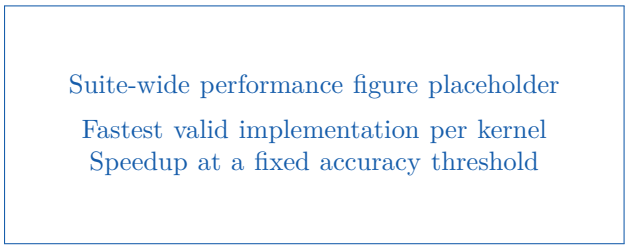


Fig. 4. Placeholder: Per-kernel speedup over a declared measured reference at a fixed accuracy threshold. Report the covered workload set and kernels with no qualifying implementation.

0.183%. All 36 paired holdouts improve, by 4.10–525× per input. Low-rank compression moves from NaNs on all 36 observations to finite errors of 0.1145–1.305%, with median 0.167%. Thus a holdout slightly above the acceptance threshold can still represent a substantial numerical improvement.

The benefit remains input-dependent. Poisson-CG moves from eight to 88 finite observations out of 96, with repaired finite median 0.579% and maximum 5.328%; all eight repairs still produce NaN/Inf on double scale. On the eight inputs where the baseline is already finite (0.0935% error), three repairs reduce error and five increase it to 0.1172–0.1907%. ADI moves from NaNs on all twelve holdouts to finite outputs throughout, but its median is 2.788% and its maximum is 27.316%, versus 0.00542% on the original fixture. Correlation moves from 60 NaN outputs to 59 finite outputs, with finite median 0.0476%; however, Terra c1 reaches 90.393% and 99.607%, and Luna c1 retains a double-scale NaN. Low typical error therefore coexists with severe outliers.

The five additional diagnostic sources remain separately labeled and outside the primary FP16 summaries. Their original semantic and baseline holds are retained. JPEG image errors are only 0.0252–0.4998%, but complete-output errors also score the MSE and count: ten or eleven repaired holdouts per source have non-finite scalar outputs, and finite MSE error reaches 100% for c2/c3. The table retains these observations alongside the BF16 DFT and held Jacobi-2D c3 diagnostics. Detailed per-input values, canonical controls, and component diagnostics accompany the frozen data.

These results support generalization of repair benefits to the tested held-out inputs, without establishing freedom from overfitting or uniform accuracy recovery. Numerical stabilization and substantial finite-error reductions remain valuable above the acceptance threshold, while non-finite outputs, large outliers, and regressions remain explicit. Shared kernels, baseline sources, and input families make pooled statistics descriptive; the experiment does not establish arbitrary-input or other-size correctness or certify implicit hardware accumulation precision.

a) Completed Luna and GPT-OSS checks: After the numerical continuation queue finished, we reran all four eligible Luna FP16 recoveries on A100 using their unchanged frozen sources and the original holdout policy. All 4 canonical controls pass; 0/4 implementations pass every holdout, with 41/48 individual holdouts passing. The decisions match the earlier evaluation. GPT-OSS contributes no recovered source to this ablation: its completed matched inventory contains zero eligible FP16 or BF16 full recoveries. An empty recovered subset is not a measured holdout failure rate. The original twenty-source FP16 comparison and separately labeled diagnostics remain unchanged.

VIII. Related Work

Recent work has progressed from benchmarking LLM-generated GPU kernels [22], [23] to execution-grounded and multi-agent optimization systems that use compiler, profiler, and runtime feedback [24]–[30]. Related efforts extend LLM-driven optimization to tensor and wafer-scale accelerators [12], [31]–[34]. These systems primarily seek a performant implementation for a fixed target or target family. Our setting instead asks which combinations of formulation, numerical format, and backend are simultaneously executable and sufficiently accurate.

Coding agents have also been used to modernize and port scientific software, including Fortran-to-C/C++, Fortran-to-Kokkos, and iterative HPC optimization workflows [35]–[38]. These efforts target a portable language or programming model; our system instead searches across accelerator-specific compiler stacks exposing partially overlapping subsets of a common tensor interface. Across these lines of work, accelerator programming, numerical tuning, and agentic optimization are typically treated separately; we search their joint space of semantically equivalent formulations, precisions, numerical repairs, and backends.

Precimonious searches floating-point precision assignments subject to an accuracy constraint, while FPTuner combines mixed-precision tuning with rigorous error analysis [39], [40]. ADAPT uses algorithmic differentiation to guide precision selection, and FPLearner predicts performance and accuracy to accelerate the search [41], [42]. More recently, Heldens and van Werkhoven jointly tune accuracy- and performance-related parameters for GPU kernels [43]. These systems primarily search numerical configurations of an existing implementation; our pipeline can also change the algorithmic formulation in response to reduced-precision failures and evaluate those transformations jointly with backend feasibility and performance.

IX. Discussion and Future Work

The paper has demonstrated the effectiveness of using Torch to write accelerator kernels outside of the machine learning domain. Moreover, it demonstrates that current AI agents are capable of translating C code into Torch, recovering its numerical error at low precision, and specializing it based on the limitations of a specific accelerator Torch SDK. Those results show that current agentic systems could help domain scientists port scientific kernels to accelerators, obtaining massive performance gains. Agents help overcome most of the programming challenges, while MLIR-based SDKs handle compilation efficiently.

Future work could expand the search of architectures by integrating other Torch-compatible accelerators, such as Meta's MTIAs, Google TPUs, and AWS Tranium accelerators.

Another interesting research opportunity is to integrate the pipeline with HLS compilers that generate FPGA accelerators starting from tensor descriptions [44]–[46].

X. Conclusion

Acknowledgment

UIC, ALCF, and all funding TODO.

References

[1] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen et al., "PyTorch: An imperative style, high-performance deep learning library," in Advances in Neural Information Processing Systems (NeurIPS), 2019.

[2] E. Lindholm, J. Nickolls, S. Oberman, and J. Montrym, "NVIDIA Tesla: A unified graphics and computing architecture," IEEE Micro, vol. 28, no. 2, pp. 39–55, 2008.

[3] J. Nickolls and W. J. Dally, "The GPU computing era," IEEE Micro, vol. 30, no. 2, pp. 56–69, 2010.

[4] Y.-H. Chen, J. Emer, and V. Sze, "Eyeriss: A spatial architecture for energy-efficient dataflow for convolutional neural networks," in Proc. of the 43rd Annual International Symposium on Computer Architecture (ISCA), 2016, pp. 367–379.

[5] H. Kwon, A. Samajdar, and T. Krishna, "MAERI: Enabling flexible dataflow mapping over DNN accelerators via reconfigurable interconnects," in Proc. of the 23rd International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2018, pp. 461–475.

[6] H. Kwon, P. Chatarasi, M. Pellauer, A. Parashar, V. Sarkar, and T. Krishna, "Understanding reuse, performance, and hardware cost of DNN dataflow: A data-centric approach," in Proc. of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO), 2019, pp. 754–768.

[7] D. Abts, J. Ross, J. Sparling, M. Wong-VanHaren, M. Baker, T. Hawkins, A. Bell et al., "Think fast: A tensor streaming processor (TSP) for accelerating deep learning workloads," in Proc. of the 47th Annual International Symposium on Computer Architecture (ISCA), 2020, vERIFY citation details.

[8] N. Brown, B. Echols, J. Zarins, and T. Grosser, "Tensorflow as a dsl for stencil-based computation on the cerebras wafer scale engine," arXiv preprint arXiv:2210.04795, 2022.

[9] R. Sai, J. Mellor-Crummey, J. Xu, and M. Araya-Polo, "Automated code generation of high-order stencils for a dataflow architecture," in SC24: International Conference for High Performance Computing, Networking, Storage and Analysis. IEEE, 2024, pp. 1–13.

[10] N. Stawinoga, D. Katz, A. Lydike, J. Zarins, N. Brown, G. Bisbas, and T. Grosser, "An mlir lowering pipeline for stencils at wafer-scale," in Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, 2026, pp. 94–109.

[11] N. J. Higham, Accuracy and Stability of Numerical Algorithms, 2nd ed. SIAM, 2002.

[12] Y. Luo, E. Huerta, V. Vishwanath, C. Ding, R. Thakur, and L. Chen, "FABRICA: Agentic CUDA-to-CSL translation and optimization for wafer-scale systems," arXiv preprint arXiv:2608.25124, 2026. [Online]. Available: https://arxiv.org/abs/2608.25124

[13] R. Chard et al., "Globus Compute: federated function-as-a-service for research computing," 2026, vERIFY – confirm authors/venue/year (funcX/Globus Compute).

[14] Globus Labs, "Academy: agents for federated and HPC execution," 2026, vERIFY – software, academy-py; confirm author/URL/version.

[15] L.-N. Pouchet and T. Yuki, "PolyBench/C: The polyhedral benchmark suite," https://sourceforge.net/projects/polybench/, 2016, vERIFY version/year.

[16] NVIDIA, "NVIDIA A100 Tensor Core GPU Architecture: Unprecedented Acceleration at Every Scale," https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/nvidia-ampere-architecture-whitepaper.pdf, 2020.

[17] Argonne Leadership Computing Facility, "Groq: System Overview," https://docs.alcf.anl.gov/ai-testbed/groq/, Accessed October 21, 2026.

[18] ——, "Cerebras: System Overview," https://docs.alcf.anl.gov/ai-testbed/cerebras/, Accessed October 21, 2026.

[19] Cerebras Systems, "CS-3 System Specifications," https://cdn.sanity.io/files/e4qjo92p/production/864133620041e5af418074842e42a669c137fa23.pdf, Vendor datasheet; sparse FP16 throughput and 27-kW maximum system power. Accessed October 21, 2026.

[20] Argonne Leadership Computing Facility, "Graphcore: System Overview," https://docs.alcf.anl.gov/ai-testbed/graphcore/, Accessed October 21, 2026.

[21] Graphcore, "Bow-2000 IPU-Machine Datasheet: Product Description," https://docs.graphcore.ai/projects/bow-2000-datasheet/en/latest/product-description.html, Accessed October 21, 2026.

[22] A. Ouyang, S. Guo, S. Arora, A. L. Zhang, W. Hu, C. Ré, and A. Mirhoseini, "Kernelbench: Can llms write efficient gpu kernels?" arXiv preprint arXiv:2502.10517, 2025.

[23] H. Wang, J. Zhang, K. Jiang, H. Wang, J. Chen, and J. Zhu, "Kernelbenchx: A comprehensive benchmark for evaluating llm-generated gpu kernels," arXiv preprint arXiv:2605.04956, 2026.

[24] Z. Zhang, R. Wang, S. Li, Y. Luo, M. Hong, and C. Ding, "Cudaforge: An agent framework with hardware feedback for cuda kernel optimization," arXiv preprint arXiv:2511.01884, 2025.

[25] K. Lei, H. Yang, H. Zhang, X. You, K. Zhang, Z. Luan, Y. Liu, and D. Qian, "Pragma: A profiling-reasoned multi-agent framework for automatic kernel optimization," arXiv preprint arXiv:2511.06345, 2025.

[26] A. Wei, T. Sun, Y. Seenichamy, H. Song, A. Ouyang, A. Mirhoseini, K. Wang, and A. Aiken, "Astra: A multi-agent system for gpu kernel performance optimization," arXiv preprint arXiv:2509.07506, 2025.

[27] J. Dong, Y. Yang, T. Liu, Y. Wang, F. Qi, V. Tarokh, K. Rangadurai, and S. Yang, "Stark: Strategic team of agents for refining kernels," in International Conference on Learning Representations, vol. 2026, 2026, pp. 20 416–20 438.

[28] Q. Sun, J. Han, T. Li, Z. Tang, S. Chen, F. Yang, A. Liu, X. Liu, and Y. Liu, "Kernelskill: A multi-agent framework for gpu kernel optimization," arXiv preprint arXiv:2603.10085, 2026.

[29] W. Dai, H. Wu, Q. Yu, H. ang Gao, J. Li, C. Jiang, W. Lou, Y. Song, H. Yu, J. Chen, W.-Y. Ma, Y.-Q. Zhang, J. Liu, M. Wang, X. Liu, and H. Zhou, "CUDA Agent: Large-scale agentic rl for high-performance cuda kernel generation," 2026.

[30] Y. Shui, C. Ma, H. Xu, S. Wen, and Y. Wang, "Harness engineering for llm-driven gpu kernel generation," 2026.

[31] C. Hong, S. Bhatia, A. Haan, S. K. Dong, D. Nikiforov, A. Cheung, and Y. S. Shao, "Llm-aided compilation for tensor accelerators," in 2024 IEEE LLM Aided Design Workshop (LAD). IEEE, 2024, pp. 1–14.

[32] C. Hong, S. Bhatia, A. Cheung, and S. Shao, "Autocomp: Llm-driven code optimization for tensor accelerators," in Machine Learning for Computer Architecture and Systems 2025, 2025.

[33] G. Zhang, S. Zhu, A. Wei, Z. Song, A. Nie, Z. Jia, N. Vijaykumar, Y. Wang, and K. Olukotun, "Accelopt: A self-improving llm agentic system for ai accelerator kernel optimization," Proceedings of Machine Learning and Systems, vol. 8, pp. 541–568, 2026.

[34] A. Hammond, A. Markosyan, A. Dontula, S. Mahns, Z. Fisches, D. Pedchenko, K. Muzumdar, N. Supper, S. Cao, H. Zhu, M. Saroufim, J. Isaacson, L. Wang, W. Hunt, K. Gondkar, R. Levenstein, G. Synnaeve, R. Li, J. Kahn, and A. Mathews, "Agentic operator generation for ml asics," in Proceedings of Machine Learning and Systems, vol. 8, 2026.

[35] P. Valero-Lara, W. F. Godoy, J. Gonzalez, A. Huante, H. Gauthier-Chaparro, J. Gonzalez, Y. K. Tang, K. Teranishi, and J. S. Vetter, "LLM-driven fortran-to-c/c++ portability for parallel scientific codes," in Proceedings of the 2025 IEEE International Conference on e-Science. IEEE, 2025, pp. 385–394.

[36] S. Gupta, K. Kamalakkannan, M. Moraru, G. Shipman, and P. Diehl, "From legacy fortran to portable kokkos: An autonomous agentic ai workflow," 2025.

[37] A. Rahman, V. Cvetkovic, K. Reece, A. Walters, Y. Hassan, A. Tummeti, B. Torres, D. Cooney, M. Ellis, and D. S. Nikolopoulos, "MARCO: A multi-agent system for optimizing hpc code generation using large language models," 2025.

[38] S. ichiro Hayashi, K. Morita, D. Mukunoki, T. Hoshino, and T. Katagiri, "VibeCodeHPC: An agent-based iterative prompting auto-tuner for hpc code generation using llms," 2025.

[39] C. Rubio-González, C. Nguyen, H. D. Nguyen, J. Demmel, W. Kahan, K. Sen, D. H. Bailey, C. Iancu, and D. Hough, "Precimonious: Tuning assistant for floating-point precision," in Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis. ACM, 2013. [Online]. Available: https://doi.org/10.1145/2503210.2503296

[40] W.-F. Chiang, M. Baranowski, I. Briggs, A. Solovyev, G. Gopalakrishnan, and Z. Rakamarić, "Rigorous floating-point mixed-precision tuning," in Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages. ACM, 2017, pp. 300–315.

[41] H. Menon, M. O. Lam, D. Osei-Kuffuor, M. Schordan, S. Lloyd, K. Mohror, and J. Hittinger, "ADAPT: Algorithmic differentiation applied to floating-point precision tuning," in Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis. IEEE / ACM, 2018, pp. 48:1–48:13.

[42] Y. Wang and C. Rubio-González, "Predicting performance and accuracy of mixed-precision programs for precision tuning," in Proceedings of the 46th IEEE/ACM International Conference on Software Engineering. ACM, 2024, pp. 15:1–15:13.

[43] S. Heldens and B. van Werkhoven, "Accuracy-aware mixed-precision gpu auto-tuning," IEEE Transactions on Parallel and Distributed Systems, vol. 37, no. 4, pp. 867–884, 2026.

[44] F. Fahim, B. Hawks, C. Herwig, J. Hirschauer, S. Jindariani, N. Tran, L. P. Carloni, G. Di Guglielmo, P. Harris, J. Krupa et al., "hls4ml: An open-source codesign workflow to empower scientific low-power machine learning devices," arXiv preprint arXiv:2103.05579, 2021.

[45] N. B. Agostini, S. Curzel, V. Amatya, C. Tan, M. Minutoli, V. G. Castellana, J. Manzano, D. Kaeli, and A. Tumeo, "An mlir-based compiler flow for system-level design and hardware acceleration," in Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design, 2022, pp. 1–9.

[46] H. Ye, C. Hao, J. Cheng, H. Jeong, J. Huang, S. Neuendorffer, and D. Chen, "Scalehls: A new scalable high-level synthesis framework on multi-level intermediate representation," in 2022 IEEE international symposium on high-performance computer architecture (HPCA). IEEE, 2022, pp. 741–755.